Understanding CUDA threadIdx

https://gemini.google.com/app/30f8dea9fda22aa9

User prompt: ThreadIdx

Response: In CUDA C/C++ programming, **`threadIdx`** is a built-in vector variable used to identify a thread's unique position within its thread block.

Structure & Dimensions

`threadIdx` is a 3D vector of type `uint3` with three components:

- `threadIdx.x`
- `threadIdx.y`
- `threadIdx.z`

The dimensions of the block containing the thread are defined by `blockDim` (`blockDim.x`, `blockDim.y`, `blockDim.z`).

Global Thread Index Calculation

Because a CUDA kernel executes across a grid of blocks, `threadIdx` is combined with `blockIdx` (the block's position in the grid) and `blockDim` to compute a thread's unique global index across the entire grid:

1D Grid & 1D Block

```
int global_id = blockIdx.x * blockDim.x + threadIdx.x;
```

2D Grid & 2D Block (e.g., Image Processing / Matrix Operations)

```
int x = blockIdx.x * blockDim.x + threadIdx.x;
int y = blockIdx.y * blockDim.y + threadIdx.y;

// Flattened row-major linear index:
int global_id = y * (gridDim.x * blockDim.x) + x;
```

3D Grid & 3D Block (e.g., Volumetric / 3D Tensor Data)

```
int x = blockIdx.x * blockDim.x + threadIdx.x;
int y = blockIdx.y * blockDim.y + threadIdx.y;
int z = blockIdx.z * blockDim.z + threadIdx.z;

int width  = gridDim.x * blockDim.x;
int height = gridDim.y * blockDim.y;

// Flattened linear index:
int global_id = z * (width * height) + y * width + x;
```

Basic Usage Example

```
#include <cuda_runtime.h>
#include <stdio.h>

__global__ void printIndexKernel() {
    int global_id = blockIdx.x * blockDim.x + threadIdx.x;

    printf("Block: %d, Local Thread: %d, Global Index: %d\n",
           blockIdx.x, threadIdx.x, global_id);
}

int main() {
    // Launch 2 blocks of 4 threads each
```

```
    printIndexKernel<<<2, 4>>>();
    cudaDeviceSynchronize();
    return 0;
}
```

Key Properties

- **Scope:** Local to the thread block. Threads in different blocks can share the same `threadIdx` values.
- **Shared Memory Indexing:** Frequently used to index into `__shared__` memory buffers allocated per block.
- **Warp Division:** Within a block, threads are grouped into warps of 32 contiguous threads based on linear order:

$$\text{Thread ID in Block} = \text{threadIdx.x} + \text{threadIdx.y} \times \text{blockDim.x} + \text{threadIdx.z} \times (\text{blockDim.x} \times \text{blockDim.y})$$
